

Associer des Informations

Présumez que nous voulons associer un nombre entier aux différents types de fruits

apple	banana	cherry	grape	mango	peach	pear
1	10	5	4	8	9	3

Comment de stocker cela?

Approche 1:

Listes de Tuples

apple	banana	cherry	grape	mango	peach	pear
1	10	5	4	8	9	3

```
[ ("Apple",1), ("Banana",10), ("Cherry",5),  
  ("Grape",4), ("Mango",8), ("Peach",9), ("Pear",3) ]
```

Approche 2:

Listes en Utilisant les Positions

apple	banana	cherry	grape	mango	peach	pear
1	10	5	4	8	9	3

	apple		pear	grape	cherry			mango	peach	banana
--	-------	--	------	-------	--------	--	--	-------	-------	--------

```
[None, "Apple", None, "Pear", "Grape",  
"Cherry", None, None, "Mango", "Peach", "Banana"]
```

Approche 2:

Listes en Utilisant les Positions

	apple		banana		cherry	grape		mango	peach	pear
--	-------	--	--------	--	--------	-------	--	-------	-------	------

Avantage:

Etant donné un identifiant, il est facile de rechercher le nom du fruit

```
print ( names[1] )
```

Désavantage:

Etant donné un nom, il est difficile de rechercher l'identifiant du fruit

```
print ( identifier["Cherry"] )
```

Dictionnaires

Structure de données qui permet de stocker et rechercher des informations associées à une valeur facilement

On peut écrire:

```
print ( identifieur["Cherry"] )
```

pour les dictionnaires!

Dictionnaires

Initialiser un dictionnaire et récupérer les informations associées

apple	banana	cherry	grape	mango	peach	pear
1	10	5	4	8	9	3

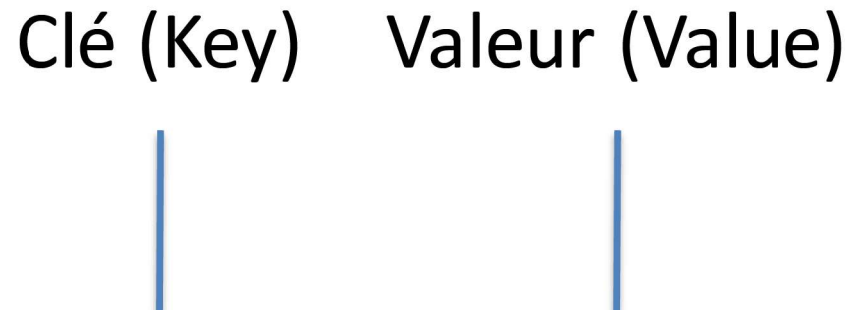
```
identifrier = { "Apple" : 1, "Banana" : 10, "Cherry" : 5, \
                "Grape" : 4, "Mango" : 8, "Peach" : 9, "Pear" : 3 }

print ( identifrier["Banana"] )
```

10

Dictionnaires

Clé (Key) Valeur (Value)



```
identifiant = { "Apple" : 1, "Banana" : 10,  
               "Cherry" : 5, "Grape" : 4, "Mango" : 8,  
               "Peach" : 9, "Pear" : 3 }  
  
print ( identifiant["Banana"] )
```

Dictionnaires

- Avantages:
 - Notation facile: tout valeur immuable peut être utilisé comme "index" (clé)
pas seulement des nombres entiers, mais aussi des chaînes de caractères, des tuples, ... et leurs combinaisons!
- ```
weird = { "Banana" : 2, (2,5) : 3, 10: 5 }
print (weird["Banana"], weird[10])
```
- Structure de données associative efficace
- Utilisés souvent en Python au lieu de listes triées



# Notation Supplémentaire

- Créer un dictionnaire vide:

```
d = { }
```

- Changer **ou** créer une association:

```
d["Apple"] = 1
```

# Manipulation de Dictionnaires

- Code correct:

```
d = { }
d["Apple"] = 1
d["Pear"] = 4
d["Pear"] = 3
print (d["Pear"])
```

{ }

{ "Apple" : 1 }

{ "Apple" : 1, "Pear" : 4 }

{ "Apple" : 1, "Pear" : 3 }

3

# Manipulation de Dictionnaires

- Code qui donne une erreur:

```
d = { }
d["Apple"] = 1
d["Pear"] = 4
d["Pear"] = 3
print (d["Grape"])
```

```
{ "Apple" : 1, "Pear" : 3 }
```

```
KeyError: 'Grape'
```

# Manipulation de Dictionnaires

Comment éviter cette erreur?

Approche 1 (défensive): tester si la valeur est présente

```
d = { }
d["Apple"] = 1
d["Pear"] = 4
d["Pear"] = 3
if "Grape" in d:
 print (d["Grape"])
```

```
{ "Apple" : 1, "Pear" : 3 }
```

**in** ici évalue seulement la présence de la clé !!!

# Manipulation de Dictionnaires

Comment éviter cette erreur?

Approche 2 (exceptions)

```
d = { }
d["Apple"] = 1
d["Pear"] = 4
d["Pear"] = 3
try:
 print (d["Grape"])
except:
 print ("Not found")
```

{ "Apple" : 1, "Pear" : 3 }

Not found

# Manipulation de Dictionnaires

Comment éviter cette erreur?

Approche 3 (get)

```
d = { }
d["Apple"] = 1
d["Pear"] = 4
d["Pear"] = 3
print (d.get("Grape", -1))
```

{ "Apple" : 1, "Pear" : 3 }

-1

Valeur retournée si la  
clé n'est pas présente

# Parcourir les Dictionnaires

- Important

Les dictionnaires n'ont pas d'ordre en Python!!!  
Les clés ne sont pas triées!!!

- Si l'ordre est important, il faut utiliser des listes

# Parcourir les Dictionnaires

- Parcourir les clés

```
identifiant = { "Apple" : 1, "Banana" : 10, "Cherry" : 5, \
 "Grape" : 4, "Mango" : 8, "Peach" : 9, "Pear" : 3 }

for key in identifiant.keys ():
 print (key)
```

Apple  
Banana  
Cherry  
Grape  
Mango  
Peach  
Pear



# Parcourir les Dictionnaires

- Parcourir les clés (simplifié)

```
identifiant = { "Apple" : 1, "Banana" : 10, "Cherry" : 5, \
 "Grape" : 4, "Mango" : 8, "Peach" : 9, "Pear" : 3 }

for key in identifiant:
 print (key)
```

Apple  
Banana  
Cherry  
Grape  
Mango  
Peach  
Pear

# Parcourir les Dictionnaires

- Parcourir les valeurs

```
identifiant = { "Apple" : 1, "Banana" : 10, "Cherry" : 5, \
 "Grape" : 4, "Mango" : 8, "Peach" : 9, "Pear" : 3 }

for value in identifiant.values ():
 print (value)
```

```
1
10
5
4
8
9
3
```

# Parcourir les Dictionnaires

- Parcourir les combinaisons (clé, valeur)

```
identifiant = { "Apple" : 1, "Banana" : 10, "Cherry" : 5, \
 "Grape" : 4, "Mango" : 8, "Peach" : 9, "Pear" : 3 }

for key, value in identifiant.items ():
 print (value)
```

```
Apple 1
Banana 10
Cherry 5
Grape 4
Mango 8
Peach 9
Pear 3
```

# Matrices

## Représentation de matrices en utilisant les listes

|   |   |   |   |
|---|---|---|---|
| 1 | 0 | 2 | 0 |
| 0 | 2 | 0 | 0 |
| 0 | 0 | 4 | 0 |
| 0 | 0 | 0 | 1 |

```
matrix = [[1, 0, 2, 0], \
 [0, 2, 0, 0], \
 [0, 0, 4, 0], \
 [0, 0, 0, 1]]
```

# Matrices

Représentation de matrices en utilisant les dictionnaires

|   |   |   |   |
|---|---|---|---|
| 1 | 0 | 2 | 0 |
| 0 | 2 | 0 | 0 |
| 0 | 0 | 4 | 0 |
| 0 | 0 | 0 | 1 |

```
matrix = { (0,0): 1, (0,2): 2, (1,1): 2, (2,2): 4, (3,3): 1 }
```

Stocke les positions pour lesquelles la valeur  $\neq 0$

Tuples sont immuables: possible comme clé

# Structures de Données Imbriquées

On peut imbriquer  
des listes/tuples dans des dictionnaires,  
des dictionnaires dans des listes/tuples,  
des dictionnaires dans des dictionnaires, ...

```
matrix = { (0,0): 1, (0,2): 2, (1,1): 2, (2,2): 4, (3,3): 1 }
```

```
nested = { "Jean" : { "Beer" : 2, "Wine" : 3 }, \
 "Valérie" : { "Wine" : 4 } }
```

```
other_nested = { "Jean" : ["Beer", "Wine"], \
 "Valérie" : ["Wine"] }
```

```
l = [{ "Beer" : 2, "Wine" : 3 }, { "Wine" : 4 }]
```

# Listes vs Dictionnaires

## Listes

- Ont d'ordre
- Chercher est seulement rapide si la liste est triée
- Chercher des informations associées nécessite de code complexe (recherche dichotomique, tuples)
- Rapide si la clé est un nombre entier

## Dictionnaires

- N'ont pas d'ordre
- Permettent de chercher rapidement
- Facile de chercher des informations associées
- Moins rapide si la clé est un nombre entier